

NYC Taxi Lakehouse: Scala + Spark Data Engineering Project

- NYC Taxi Lakehouse: Scala + Spark Data Engineering Project
 - 项目简介
 - 适合人群
 - 学习目标
 - 推荐技术栈
 - 环境配置建议
 - 推荐主环境：本地 Scala + Spark
 - Mac 本地安装示例
 - 可选入门环境：Google Colab
 - 可选展示环境：Databricks
 - 平台选择与成本说明
 - 数据集
 - 项目背景
 - 建议目录结构
 - 数据湖分层要求
 - 1. Raw Layer
 - 2. Cleaned Layer
 - 3. Curated Layer
 - 4. Analytics Layer
 - 任务清单
 - Task 1: 项目初始化
 - Task 2: 数据摄取
 - Task 3: 数据质量检查
 - Task 4: 数据清洗
 - Task 5: 维度建模
 - Task 6: 分析指标
 - Task 7: Spark SQL 与 DataFrame API 对比
 - Task 8: 性能优化实验
 - Task 9: 测试
 - Task 10: 项目文档

- 加分项
 - Bonus 1: Streaming 模拟
 - Bonus 2: Delta Lake
 - Bonus 3: Airflow 编排
 - Bonus 4: 简单 Dashboard
 - Bonus 5: 云端版本
- 评分标准
- 推荐完成节奏
 - Week 1
 - Week 2
 - Week 3
 - Week 4
- 面试延展问题
- 最终交付物
- 简历描述示例

NYC Taxi Lakehouse: Scala + Spark Data Engineering Project

项目简介

这个项目面向想练习 Data Engineering 的学习者，目标是用 Scala 和 Apache Spark 处理一个真实的大型公开数据集，完成从原始数据摄取、清洗、建模、聚合到性能优化的一整套数据工程流程。

推荐数据集为 NYC Taxi & Limousine Commission Trip Record Data。该数据集包含纽约出租车和网约车行程记录，字段包括上下车时间、上下车区域、行程距离、费用、支付方式、乘客数量等。数据量可以按月份自由选择，既能在本地练习，也能扩展到云端或集群环境。

适合人群

- 已经学过 Scala 基础语法
- 想巩固 Spark DataFrame、Dataset、Spark SQL
- 想理解 Data Engineer 日常工作中的 ETL、数据质量、分区、建模和性能优化
- 想做一个可以写进简历或 GitHub Portfolio 的项目

学习目标

完成项目后，学习者应该能够：

- 使用 Scala + Spark 读取大规模 CSV / Parquet 数据
- 设计基础的数据湖分层：raw、cleaned、curated、analytics
- 编写可重复运行的 batch pipeline
- 处理脏数据、异常值、重复数据和 schema 问题
- 使用 Spark SQL / DataFrame API 完成聚合分析
- 将结果以 Parquet 格式按日期或月份分区落盘
- 设计简单的事实表和维度表
- 使用 explain plan、repartition、cache 等方法分析和优化 Spark 作业
- 编写基础测试和项目文档

推荐技术栈

- Language: Scala
- Processing: Apache Spark
- Build Tool: sbt
- Storage: Local filesystem, optionally S3-compatible storage
- Data Format: CSV, Parquet
- Optional: Docker, Airflow, DuckDB, Trino, PostgreSQL, Delta Lake
- Testing: ScalaTest

环境配置建议

本项目建议采用“本地 Scala 工程为主，Colab 或 Databricks 为辅助”的方式完成。

推荐路线：

1. 先用 Colab 或 notebook 快速熟悉数据字段和 Spark 操作
2. 正式项目使用本地 Scala + sbt + Spark 项目完成
3. 项目跑通后，再迁移到 Databricks 做平台化展示

推荐主环境：本地 Scala + Spark

本地环境最适合练习 Data Engineer 的工程能力，包括项目结构、依赖管理、测试、配置、GitHub 提交和可重复运行的 batch job。

推荐版本组合：

组件	推荐版本	说明
Java	17 LTS	Spark 3.5.x 支持 Java 8/11/17； Java 17 更适合长期学习
Scala	2.12.x	与 Spark 3.5.x 的常见发行版本兼容
sbt	1.9+	Scala 项目构建工具
Spark	3.5.x	稳定、资料多，适合学习 Scala Spark
IDE	IntelliJ IDEA 或 VS Code	IntelliJ 对 Scala/sbt 体验更完整

参考：

- Apache Spark 3.5.1 documentation
<https://archive.apache.org/dist/spark/docs/3.5.1/>
- Scala JDK compatibility
<https://docs.scala-lang.org/overviews/jdk-compatibility/overview.html>

Mac 本地安装示例

如果使用 macOS，可以用 Homebrew 安装基础工具：

```
brew install openjdk@17
brew install scala
brew install sbt
```

检查版本：

```
java -version
scala -version
sbt --version
```

项目依赖建议放在 `build.sbt` 中，而不是手动下载 Spark：

```
ThisBuild / scalaVersion := "2.12.18"

libraryDependencies ++= Seq(
  "org.apache.spark" %% "spark-core" % "3.5.1",
  "org.apache.spark" %% "spark-sql" % "3.5.1",
  "org.scalatest" %% "scalatest" % "3.2.18" % Test
)
```

最小 SparkSession 示例：

```
import org.apache.spark.sql.SparkSession

object Main {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder()
      .appName("nyc-taxi-lakehouse")
      .master("local[*]")
      .getOrCreate()

    spark.sparkContext.setLogLevel("WARN")

    import spark.implicits._
    Seq(("hello", 1)).toDF("word", "count").show()

    spark.stop()
  }
}
```

可选入门环境：Google Colab

Colab 适合作为第一天的快速体验环境，尤其适合用 PySpark 先理解数据字段、Spark DataFrame、Spark SQL 和基础聚合。

适合做：

- 读取 1 个月 NYC Taxi 数据
- 快速 `filter`、`groupBy`、`join`
- 熟悉 Spark SQL
- 做探索性分析和截图

不建议作为主项目环境，因为 Colab 默认是 Python notebook 工作流，不适合练 Scala 工程项目、sbt、测试和打包。Colab 免费资源也不是固定承诺资源，可能出现 runtime timeout 或资源限制。

Colab PySpark 快速启动示例：

```
!pip install pyspark

from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("nyc-taxi-colab") \
    .getOrCreate()
```

参考：

- Google Colab FAQ
<https://research.google.com/colaboratory/faq.html>

可选展示环境：Databricks

Databricks 适合在项目后半段使用，用来展示更接近真实工作的 lakehouse / notebook / job / table 工作流。

适合做：

- 上传或挂载 taxi 数据
- 用 Databricks notebook 跑 Spark
- 创建临时表或 managed table
- 展示 Spark SQL 查询结果
- 对比本地 Spark 与云端 Spark 平台体验

不建议一开始完全依赖 Databricks，因为 notebook 平台会掩盖一部分工程训练，例如 sbt 项目结构、测试、打包和本地调试。

参考：

- Databricks Free Edition
<https://docs.databricks.com/aws/en/getting-started/free-edition>
- Databricks for Scala developers
<https://docs.databricks.com/aws/en/languages/scala>

平台选择与成本说明

本项目可以在免费或低成本环境完成。建议把成本控制作为项目要求之一：先用小数据集验证逻辑，再扩大数据量。

环境	是否推荐	成本倾向	适合阶段	主要注意事项
本地电脑	强烈推荐	免费，使用个人电脑资源	正式主项目	需要安装 Java、Scala、sbt；本地内存有限时先用 1-3 个月数据
Google Colab	可选	免费层可用，资源不保证	快速入门、PySpark 探索	更适合 Python/PySpark；不适合作为 Scala 工程主环境
Databricks Free Edition	推荐作为展示	官方提供个人学习免费版	项目展示、平台体验	免费版能力和限制可能变化，注册后需确认 Scala 和 job 支持情况
Databricks Trial / 云服务	不建议初学阶段使用	可能产生云资源费用	高阶扩展	需要设置预算提醒，避免长时间运行集群
AWS EMR / GCP Dataproc / Azure Synapse	不建议初学阶段使用	可能产生明显费用	简历进阶项目	适合已有云经验后再做

成本建议：

- 第一版只处理 1 个月 Yellow Taxi 数据
- 本地运行时使用 `local[*]`
- 输出 Parquet 时注意文件数量，避免产生大量小文件
- 云端平台不要长时间保持集群运行
- 如果使用付费云资源，必须设置预算提醒或 spending limit
- README 中记录使用了哪些环境、数据量和大概运行时间

数据集

主数据集：

- NYC TLC Trip Record Data
<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>

可选镜像或云端版本：

- NYC TLC Trip Record Data on AWS Open Data
<https://registry.opendata.aws/nyc-tlc-trip-records-pds/>

建议初始范围：

- 入门版本：1 个月 Yellow Taxi 数据
- 标准版本：6-12 个月 Yellow Taxi 数据
- 进阶版本：Yellow + Green Taxi 多年数据
- 高阶版本：Taxi 数据 + Taxi Zone Lookup + 天气数据或节假日数据

项目背景

纽约市交通分析团队希望建立一个可重复运行的数据处理 pipeline，用来分析出租车出行需求、收入趋势、区域热点和异常行程。你需要从公开的原始出租车行程数据出发，构建一套 clean、reliable、analytics-ready 的数据资产。

最终结果应该能回答这些问题：

- 每日和每月的总行程数是多少？
- 每日和每月的总收入、平均车费、平均小费是多少？
- 哪些区域是最热门的上车点和下车点？
- 哪些路线最赚钱？
- 工作日和周末的出行模式有什么差异？
- 哪些记录可能是脏数据或异常数据？
- 分区、文件格式和 repartition 对查询性能有什么影响？

建议目录结构

```
nyc-taxi-lakehouse/  
  README.md  
  build.sbt  
  project/  
  data/  
    raw/  
    cleaned/  
    curated/  
    analytics/  
  src/  
    main/  
      scala/  
        com/example/taxi/  
          Main.scala  
          config/  
          jobs/  
          models/  
          quality/  
          utils/  
    test/  
      scala/  
        com/example/taxi/  
  notebooks/  
  docs/  
    data-dictionary.md  
    architecture.md  
    findings.md
```

数据湖分层要求

1. Raw Layer

保存原始下载数据，不做修改。

要求：

- 按数据源和年月组织文件
- 保留原始字段

- 记录下载来源和时间

示例路径：

```
data/raw/yellow_taxi/year=2024/month=01/  
data/raw/taxi_zone_lookup/
```

2. Cleaned Layer

保存经过基础清洗的数据。

清洗规则建议：

- 去除 pickup time 或 dropoff time 为空的记录
- 去除 trip distance 小于等于 0 的记录
- 去除 fare amount 小于 0 的记录
- 去除 passenger count 小于等于 0 或明显异常的记录
- 计算 trip duration minutes
- 过滤 duration 小于等于 0 的记录
- 可选：过滤 duration 超过 6 小时的极端异常记录

3. Curated Layer

保存建模后的事实表和维度表。

建议表：

- fact_trips
- dim_date
- dim_zone
- dim_payment_type
- dim_rate_code

4. Analytics Layer

保存面向分析查询的聚合结果。

建议表：

- daily_trip_summary
- monthly_revenue_summary
- hourly_demand_summary

- `top_pickup_zones`
- `top_routes`
- `anomaly_trips`

任务清单

Task 1: 项目初始化

目标：搭建一个可运行的 Scala + Spark 项目。

要求：

- 使用 sbt 创建项目
- 配置 Spark SQL 依赖
- 创建一个 main entry point
- 能够本地运行一个 Spark job

交付物：

- `build.sbt`
- `Main.scala`
- README 中的运行命令

Task 2: 数据摄取

目标：读取 NYC Taxi 原始数据。

要求：

- 支持读取一个或多个月份的数据
- 支持 CSV 或 Parquet 格式
- 显式定义 schema，避免完全依赖 inferSchema
- 输出原始数据的 row count、column count 和 sample rows

思考问题：

- 为什么大数据场景下不推荐长期依赖 inferSchema?
- CSV 和 Parquet 在 Spark 中的读取差异是什么?

Task 3: 数据质量检查

目标：输出基础数据质量报告。

要求统计：

- 总行数
- 每个关键字段的 null count
- 重复记录数量
- trip distance 小于等于 0 的数量
- fare amount 小于 0 的数量
- pickup time 晚于 dropoff time 的数量
- 异常 passenger count 数量

交付物：

- data_quality_report
- README 中解释发现了哪些数据问题

Task 4: 数据清洗

目标：生成 cleaned trips 表。

要求：

- 应用 Task 3 中定义的清洗规则
- 新增字段：
 - pickup_date
 - pickup_hour
 - trip_duration_minutes
 - total_amount_per_mile
 - tip_percentage
- 输出为 Parquet
- 按 pickup_date 或 year/month 分区

交付物：

- data/cleaned/yellow_taxi/
- 清洗前后 row count 对比

Task 5: 维度建模

目标：将 cleaned 数据整理成简单的星型模型。

要求：

- 构建 fact_trips

- 构建 `dim_date`
- 使用 Taxi Zone Lookup 构建 `dim_zone`
- 可选: 构建 `dim_payment_type`

事实表建议字段:

- `trip_id`
- `pickup_datetime`
- `dropoff_datetime`
- `pickup_date`
- `pickup_hour`
- `pickup_location_id`
- `dropoff_location_id`
- `passenger_count`
- `trip_distance`
- `fare_amount`
- `tip_amount`
- `total_amount`
- `trip_duration_minutes`

交付物:

- `data/curated/fact_trips/`
- `data/curated/dim_date/`
- `data/curated/dim_zone/`

Task 6: 分析指标

目标: 基于 curated 数据生成 analytics-ready 聚合表。

必须完成的指标:

- 每日总行程数
- 每日总收入
- 每日平均 fare
- 每日平均 trip distance
- 每日平均 trip duration
- 每小时行程需求分布
- pickup zone Top 10
- dropoff zone Top 10

- route Top 10: pickup zone + dropoff zone
- 小费比例最高的区域

交付物:

- `daily_trip_summary`
- `hourly_demand_summary`
- `top_pickup_zones`
- `top_routes`

Task 7: Spark SQL 与 DataFrame API 对比

目标: 用两种方式实现同一个指标。

要求:

- 用 DataFrame API 实现每日收入聚合
- 用 Spark SQL 实现每日收入聚合
- 对比代码可读性和执行计划

交付物:

- 两份实现代码
- `explain()` 输出截图或文字说明

Task 8: 性能优化实验

目标: 观察 Spark job 在不同策略下的性能差异。

至少完成两个实验:

- CSV vs Parquet 读取时间对比
- 未分区 vs 按日期分区查询时间对比
- 不同 repartition 数量对输出文件数量和运行时间的影响
- cache 前后重复查询时间对比

交付物:

- 性能实验表格
- 对实验结果的解释

示例表格:

实验	数据量	策略 A	策略 B	结果	结论
读取格式	1 个月	CSV	Parquet	Parquet 更快	列式存储适合分析查询

Task 9: 测试

目标：为核心转换逻辑编写测试。

要求：

- 至少 3 个单元测试
- 使用小样本数据验证清洗逻辑
- 测试异常值是否被过滤
- 测试派生字段是否计算正确

交付物：

- `src/test/scala/...`
- README 中说明如何运行测试

Task 10: 项目文档

目标：写一份清晰的 README。

README 至少包含：

- 项目背景
- 技术栈
- 数据来源
- 架构图或文字说明
- 如何下载数据
- 如何运行 pipeline
- 输出表说明
- 数据质量发现
- 性能优化实验结果
- 未来可以改进的地方

加分项

Bonus 1: Streaming 模拟

将历史 taxi 数据按时间顺序模拟成流式输入，用 Spark Structured Streaming 读取并实时计算：

- 每 5 分钟行程数
- 每 5 分钟收入
- 当前最热门 pickup zone

Bonus 2: Delta Lake

将 cleaned 和 curated layer 改为 Delta Lake 表。

练习点：

- schema evolution
- upsert / merge
- time travel
- partition management

Bonus 3: Airflow 编排

使用 Airflow 编排任务：

- download data
- run quality check
- clean data
- build curated tables
- build analytics tables

Bonus 4: 简单 Dashboard

用任意工具展示最终分析结果：

- Spark SQL + notebook
- DuckDB + Parquet
- Streamlit
- Superset
- Metabase

Bonus 5: 云端版本

将项目迁移到云端：

- S3 / GCS / ADLS 存储数据
- EMR / Dataproc / Databricks 运行 Spark
- Athena / BigQuery / Trino 查询结果

评分标准

总分 100 分。

模块	分数
项目结构清晰，可运行	10
数据摄取正确	10
数据质量检查完整	15
数据清洗逻辑合理	15
分层和建模设计	15
分析指标完成度	15
Spark 性能优化实验	10
测试和文档	10

推荐完成节奏

Week 1

- 搭建 Scala + Spark 项目
- 下载 1 个月数据
- 完成数据读取和基础探索

Week 2

- 完成数据质量检查
- 完成 cleaned layer

- 写基础测试

Week 3

- 完成 curated layer 和 analytics layer
- 完成核心分析指标

Week 4

- 做性能优化实验
- 补充 README
- 可选完成一个 bonus

面试延展问题

完成项目后，可以让学习者回答这些问题：

- Spark 的 lazy evaluation 是什么？这个项目里哪里体现了？
- DataFrame、Dataset、RDD 有什么区别？
- 为什么 Parquet 通常比 CSV 更适合分析场景？
- partition 和 repartition 的区别是什么？
- 什么情况下 repartition 会让任务变慢？
- 如何判断一个 Spark job 是否发生了 shuffle？
- 数据质量检查应该放在 pipeline 的哪个阶段？
- 如果每天有新增 taxi 数据，pipeline 怎么设计成增量处理？
- 如果 schema 发生变化，应该怎么处理？
- 如果数据量增长 100 倍，哪些地方最可能先出问题？

最终交付物

学习者最终应该提交：

- GitHub repo
- 可运行的 Scala + Spark pipeline
- 处理后的 Parquet 数据样例
- README
- 数据质量报告
- 性能实验结果

- 至少 3 个测试
- 可选 dashboard 或 notebook

简历描述示例

可以写成：

Built a Scala and Apache Spark batch data pipeline for NYC Taxi trip records, processing multi-month trip data into a layered lakehouse structure. Implemented data quality validation, Parquet partitioning, dimensional modeling, analytics aggregations, and Spark performance experiments including partitioning, caching, and file format comparison.